

Projet Web Sémantique

Construction et intégration d'un graphe RDF

Auteurs :

Mohand Arezki BRANECI

Rabah ACHOUR

Année universitaire 2025–2026

Table des matières

1	Introduction	2
2	Jeux de données et sources	2
2.1	Jeux de données des pays (Kaggle)	2
2.2	Jeu de données des villes (Kaggle)	3
2.3	Jeu de données des aéroports (Kaggle)	3
2.4	Jeu de données géographiques complémentaires (Wikidata)	3
3	Transformation des données en RDF	4
3.1	Transformation du jeu de données des villes	4
3.2	Transformation du jeu de données des aéroports	4
3.3	Enrichissement des données des pays	5
4	Modélisation du graphe RDF	5
5	Intégration et alignement des données	6
5.1	Approche d'alignement manuel	6
5.2	Alignement automatique basé sur les données enrichies	7
6	Raisonnement RDFS	8
6.1	Fusion des graphes RDF	8
6.2	Définition de l'ontologie RDFS	9
6.3	Application du raisonnement RDFS	10
6.4	Optimisation des performances avec Jena TDB2	11
7	Requêtes SPARQL	11
7.1	Requête fédérée	12
7.2	Requêtes avec expressions de chemin	13
7.3	Requêtes utilisant OPTIONAL	14
7.4	Requêtes sur graphes nommés	15
7.5	Requêtes avec filtrage avancé	16
7.6	Requêtes avec raisonnement RDFS	17
8	Conclusion	19

1 Introduction

Le Web sémantique vise à structurer les données de manière à les rendre compréhensibles et exploitables par des machines. Dans ce cadre, le Resource Description Framework (RDF) permet de représenter des connaissances sous forme de graphes, où les entités sont reliées entre elles par des relations sémantiques explicites.

L’objectif de ce projet est de construire un graphe RDF intégré à partir de plusieurs jeux de données hétérogènes décrivant des villes, des pays et des aéroports. Ces données sont initialement disponibles sous différents formats (CSV et RDF) et proviennent de sources distinctes. Le travail consiste à transformer les données tabulaires en RDF, à définir une modélisation cohérente, puis à intégrer l’ensemble des données dans un graphe unique interrogeable via SPARQL.

Pour cela, nous utilisons l’outil *Tarql* afin de convertir les fichiers CSV en RDF, ainsi qu’Apache Jena pour l’exécution des requêtes SPARQL. Une attention particulière est portée à la cohérence des identifiants (URI) et à l’alignement des ressources issues de différentes sources, notamment par l’utilisation de liens `owl:sameAs`.

La suite de ce rapport présente successivement les jeux de données utilisés, les étapes de transformation en RDF, la modélisation du graphe, l’intégration et l’alignement des données, ainsi que les requêtes SPARQL finales.

Vous trouverez l’ensemble des fichiers de code dans l’url GITHUB suivante :

<https://github.com/Ravvah/semantic-web-geo-project>

2 Jeux de données et sources

Ce projet repose sur plusieurs jeux de données ouverts décrivant des entités géographiques (villes, pays et aéroports). Les données proviennent de sources hétérogènes et sont fournies sous différents formats (CSV et RDF), ce qui permet d’illustrer un scénario réaliste d’intégration de données dans le cadre du Web sémantique.

2.1 Jeux de données des pays (Kaggle)

Les données relatives aux pays proviennent d’un jeu de données publié sur la plateforme Kaggle, contenant des informations issues de DBpedia. Ce jeu de données est fourni au format CSV et associe chaque pays (identifié par une URI DBpedia) à des indicateurs supplémentaires, tels que des indices de recherche et d’inflation.

- Nom du dataset : *DBpedia Country Information*
- Source : <https://www.kaggle.com/datasets/group16/dbpedia-country-information>
- Format : CSV
- Licence : Open Data (usage académique autorisé)

Ce jeu de données a été utilisé pour enrichir les ressources DBpedia décrivant les pays. Les données ont été converties en RDF et stockées dans le fichier `countries_enriched.ttl`, qui constitue la base de référence pour les pays dans le graphe RDF final.

2.2 Jeu de données des villes (Kaggle)

Les données relatives aux villes proviennent d'un jeu de données public hébergé sur Kaggle, décrivant les principales villes du monde. Ce jeu de données est fourni au format CSV et contient notamment le nom des villes, leur population, leurs coordonnées géographiques et le pays associé.

- Nom du dataset : *World Cities Database*
- Source : <https://www.kaggle.com/datasets/juanmah/world-cities>
- Format : CSV
- Licence : Open Data (usage académique autorisé)

Ces données ont été transformées en RDF à l'aide de l'outil Tarql, et constituent le jeu de données principal pour la modélisation des entités de type `City` dans le graphe RDF.

2.3 Jeu de données des aéroports (Kaggle)

Les données relatives aux aéroports proviennent également d'un jeu de données ouvert publié sur la plateforme Kaggle. Ce jeu de données décrit les aéroports du monde entier et fournit des informations telles que le nom de l'aéroport, son type, ses codes IATA et ICAO lorsqu'ils sont disponibles, ainsi que ses coordonnées géographiques et la ville associée.

- Nom du dataset : *Airport Database*
- Source : <https://www.kaggle.com/datasets/marcslaughter/airport>
- Format : CSV
- Licence : Open Data (usage académique autorisé)

Ce jeu de données a été transformé en RDF et stocké dans le fichier `airports.ttl`. Il permet de relier les entités de type `Airport` aux entités de type `City` dans le graphe RDF.

2.4 Jeu de données géographiques complémentaires (Wikidata)

Un jeu de données RDF contenant des coordonnées géographiques a été utilisé à titre complémentaire. Ces données proviennent de Wikidata et ont été téléchargées sous forme de dump RDF via le site DBpedia.

- Source : <https://downloads.dbpedia.org/2015-04/ext/wikidata/>
- Dataset : `geo-coordinates_wikidata.ttl`
- Format : RDF (Turtle)
- Licence : Creative Commons CC0 (Wikidata)

Ce jeu de données illustre l'utilisation de sources RDF externes et la possibilité d'aligner des entités géographiques à l'aide d'identifiants Wikidata. Il est utilisé principalement à des fins d'illustration et de complément, l'intégration principale du projet reposant sur les données issues de Kaggle et de DBpedia.

3 Transformation des données en RDF

Les jeux de données utilisés dans ce projet sont initialement disponibles sous forme de fichiers CSV. Afin de les intégrer dans un graphe RDF interopérable, il est nécessaire de transformer ces données tabulaires en triplets RDF. Pour cette étape, l'outil *Tarql* (*Transformation SPARQL for RDF*) a été utilisé.

Tarql permet d'exécuter des requêtes SPARQL directement sur des fichiers CSV. Contrairement à une requête **SELECT**, qui sert uniquement à interroger des données, une requête **CONSTRUCT** permet de générer de nouveaux triplets RDF à partir des données sources. Cette fonctionnalité est essentielle pour convertir des fichiers CSV en représentations RDF structurées.

3.1 Transformation du jeu de données des villes

Le fichier `cities.csv` a été transformé en RDF à l'aide d'une requête **CONSTRUCT**. Chaque ligne du fichier CSV est convertie en une ressource de type **City**. Des URI propres sont générées pour chaque ville, et plusieurs propriétés sont associées, telles que la population, les codes ISO du pays, le statut administratif, ainsi que les coordonnées géographiques.

Une relation sémantique explicite **locatedIn** est également créée afin de relier chaque ville au pays dans lequel elle est située. Le résultat de cette transformation est stocké dans un fichier `cities.ttl` au format Turtle. La commande exécutée est :

```
tarql cities_construct.rq cities.csv > cities.ttl
```

3.2 Transformation du jeu de données des aéroports

Le fichier `airports.csv` a également été transformé en RDF à l'aide de Tarql. Chaque aéroport est modélisé comme une entité de type **Airport**. Les URI des aéroports sont générées à partir d'identifiants stables fournis dans le fichier CSV, afin d'éviter les problèmes liés aux caractères spéciaux présents dans les noms.

Les aéroports sont enrichis par des propriétés telles que le nom, le type, les codes IATA et ICAO lorsqu'ils sont disponibles, ainsi que les coordonnées géographiques. Chaque aéroport est relié à une ville via la relation **locatedIn**, et chaque ville issue du jeu de données des aéroports est à son tour reliée à un pays à l'aide du code ISO du pays.

Le résultat de cette transformation est enregistré dans le fichier `airports.ttl`, avec la commande :

```
tarql airports_construct.rq airports.csv > airports.ttl
```

3.3 Enrichissement des données des pays

Les données relatives aux pays reposent sur deux sources complémentaires : un graphe RDF existant (`countries.ttl`) et un fichier tabulaire (`countries.csv`). Le fichier RDF, issu d'une source de type Linked Open Data (DBpedia), constitue la base principale pour la description des entités de type `Country`.

Le fichier CSV contient des informations additionnelles, telles que des indicateurs de recherche et d'inflation, qui ne sont pas présentes dans le graphe RDF initial. Plutôt que de recréer de nouvelles ressources, ces données sont utilisées pour enrichir les entités RDF existantes.

Cette étape d'enrichissement est réalisée à l'aide de Tarql, en exécutant une requête SPARQL de type `CONSTRUCT` qui ajoute de nouvelles propriétés aux ressources `Country` déjà définies dans `countries.ttl`. Le résultat de cette opération est stocké dans le fichier `countries_enriched.ttl`, qui contient à la fois les triplets initiaux et les triplets ajoutés.

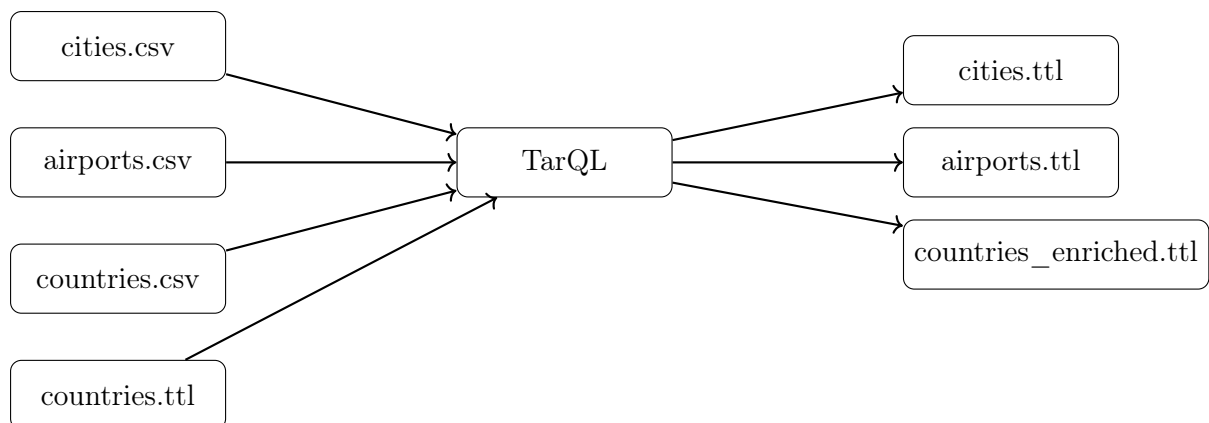


FIGURE 1 – Transformation et enrichissement des données en RDF à l'aide de Tarql

4 Modélisation du graphe RDF

La modélisation du graphe RDF repose sur trois types d'entités principales : les villes (`City`), les pays (`Country`) et les aéroports (`Airport`). Ces entités sont reliées entre elles par des relations sémantiques explicites, permettant de représenter une hiérarchie géographique cohérente.

Les villes sont modélisées comme des entités de type `City`. Chaque ville est associée à un pays par la relation `locatedIn`. Les pays sont modélisés comme des entités distinctes, ce qui permet de les enrichir et de les aligner avec des sources externes.

Les aéroports sont modélisés comme des entités de type **Airport**. Chaque aéroport est relié à une ville par la relation `locatedIn`. Afin d’assurer une cohérence globale du graphe, les villes issues du jeu de données des aéroports sont également reliées à un pays à l’aide des codes ISO des pays.

Cette modélisation hiérarchique permet de naviguer dans le graphe depuis un aéroport vers une ville, puis vers un pays, et constitue une base solide pour l’intégration de sources externes et l’exécution de requêtes SPARQL complexes.

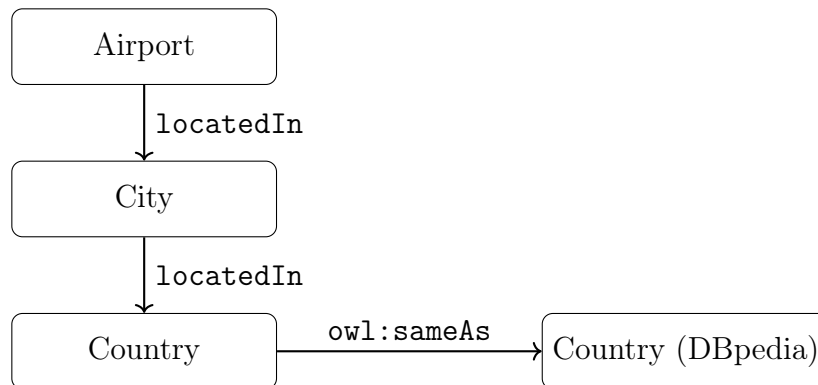


FIGURE 2 – Schéma conceptuel du graphe RDF construit

5 Intégration et alignement des données

L’intégration de sources de données hétérogènes constitue un enjeu central du Web sémantique. Dans ce projet, les données locales (villes et aéroports) sont enrichies et mises en relation avec des données issues de DBpedia. Cette intégration repose sur un mécanisme d’alignement sémantique, permettant d’indiquer que deux ressources provenant de graphes différents représentent la même entité du monde réel.

En RDF, cet alignement est exprimé à l’aide de la propriété `owl:sameAs`. Un triplet de la forme `A owl:sameAs B` signifie que les ressources A et B désignent exactement la même entité, et peuvent donc être utilisées de manière interchangeable dans les requêtes SPARQL.

5.1 Approche d’alignement manuel

Dans un premier temps, un alignement manuel minimal a été envisagé. Cette approche consiste à relier explicitement certaines ressources locales de type **Country** aux ressources correspondantes de DBpedia. L’alignement manuel présente l’avantage de garantir la validité sémantique des liens, dans la mesure où chaque correspondance est vérifiée par l’utilisateur.

Cette méthode est toutefois limitée en termes de passage à l'échelle et n'est pas adaptée lorsque le nombre de ressources à aligner devient important. Elle est néanmoins parfaitement acceptable dans un cadre pédagogique, et permet d'illustrer clairement le principe de l'alignement sémantique.

5.2 Alignement automatique basé sur les données enrichies

Afin d'aligner automatiquement l'ensemble des pays présents dans le projet, une approche basée sur les données existantes a été mise en œuvre. Le fichier `countries_enriched.ttl` contient déjà des ressources DBpedia officielles pour chaque pays, enrichies avec des indicateurs supplémentaires. Les ressources locales des pays sont quant à elles utilisées dans le fichier `cities.ttl` via la relation `locatedIn`.

L'alignement automatique repose sur l'hypothèse suivante : les identifiants des ressources locales des pays ont été construits de manière cohérente avec les identifiants DBpedia correspondants. Il est alors possible d'établir automatiquement des liens `owl:sameAs` en comparant les noms des URI locales et celles de DBpedia.

La requête SPARQL suivante, de type `CONSTRUCT`, permet de générer automatiquement un fichier d'alignement à partir des fichiers `cities.ttl` et `countries_enriched.ttl`.

```
PREFIX ex: <http://example.org/schema/>
PREFIX res: <http://example.org/resource/>
PREFIX owl: <http://www.w3.org/2002/07/owl#>

CONSTRUCT {
  ?localCountry owl:sameAs ?dbCountry .
}
WHERE {
  ?city ex:locatedIn ?localCountry .
  ?dbCountry ex:researchRating ?rating .

  FILTER(
    STRAFTER(STR(?localCountry), "http://example.org/resource/")
    =
    STRAFTER(STR(?dbCountry), "http://dbpedia.org/resource/")
  )
}
```

Cette requête parcourt l'ensemble des pays locaux utilisés dans le jeu de données des villes, ainsi que les pays présents dans le fichier enrichi issu de DBpedia. La clause `FILTER` permet d'identifier les correspondances entre ressources locales et ressources DBpedia en

comparant la partie terminale de leurs URI. Pour chaque correspondance détectée, un triplet `owl:sameAs` est généré.

Les triplets produits sont stockés dans un fichier RDF distinct, chargé au moment de l'interrogation du graphe. Ce choix permet de séparer les données sources de la logique d'intégration sémantique, et de comparer facilement les résultats avec ou sans alignement.

6 Raisonnement RDFS

Le raisonnement RDFS constitue une étape centrale du projet, permettant d'enrichir automatiquement le graphe RDF à partir de règles d'inférence définies dans une ontologie. Avant l'application de ces règles, il est nécessaire de regrouper l'ensemble des graphes RDF produits et des liens d'alignement dans un graphe unique.

Il est important de noter que l'outil `sparql` fourni par Apache Jena CLI en ligne de commande ne permet pas, dans notre configuration, d'activer directement le raisonnement RDFS lors de l'exécution des requêtes SPARQL. Autrement dit, les requêtes exécutées avec l'option `-data` n'exploitent que les triplets explicitement présents dans les fichiers RDF, sans tenir compte des règles d'inférence définies dans l'ontologie.

Afin de pouvoir appliquer le raisonnement RDFS, nous avons donc procédé à une étape préalable de fusion et d'enrichissement du graphe RDF. Cette étape consiste à combiner explicitement les données RDF groupées (villes, aéroports, pays enrichis, alignements) avec l'ontologie RDFS, en utilisant l'outil `riot` de Jena et son mode de traitement RDFS. Nous verrons ça dans la deuxième sous-section. Nous devons d'abord fusionner les RDF.

6.1 Fusion des graphes RDF

Une fois les graphes RDF produits lors des étapes de transformation et le fichier d'alignement généré, l'ensemble des données est fusionné afin d'obtenir un graphe unique. Cette étape rassemble les graphes issus des transformations (villes, aéroports et pays enrichis), ainsi que les liens d'alignement exprimés à l'aide de `owl:sameAs`.

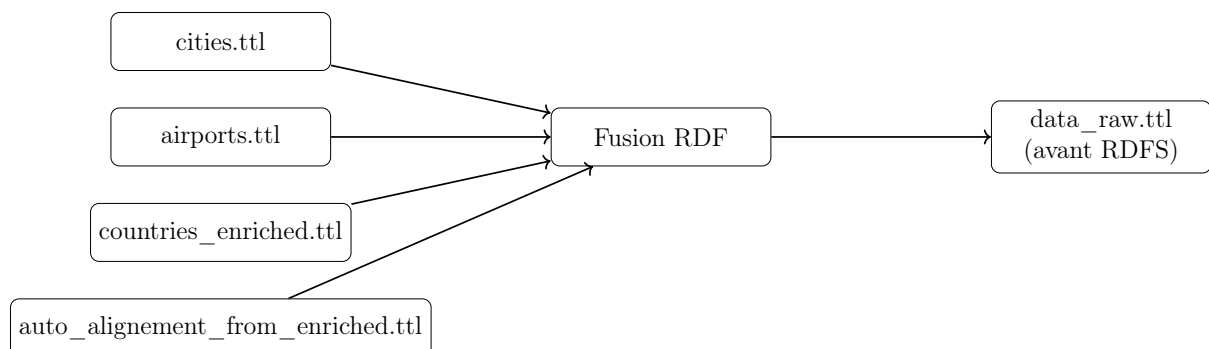


FIGURE 3 – Fusion des graphes RDF et production du graphe `data_raw.ttl` (avant raisonnement)

La fusion est réalisée à l'aide des outils en ligne de commande d'Apache Jena, en concaténant l'ensemble des fichiers RDF dans un graphe unique. Le résultat correspond au graphe *avant raisonnement*, stocké dans le fichier `data_raw.ttl`.

La commande utilisée est la suivante :

```
riot --output=ttl \  
  cities/cities.ttl \  
  airports/airports.ttl \  
  countries/countries_enriched.ttl \  
  alignement/auto_alignement_from_enriched.ttl \  
  geo_coordinates/geo-coordinates_wikidata.ttl \  
> data_raw.ttl
```

Ce fichier contient uniquement les triplets explicitement définis dans les données sources et les fichiers d'alignement, sans aucune inférence.

6.2 Définition de l'ontologie RDFS

Afin de permettre le raisonnement, une ontologie RDFS minimale a été définie dans le fichier `ontology.ttl`. Cette ontologie introduit une hiérarchie de classes et des contraintes de domaine et de portée nécessaires à l'inférence automatique de nouveaux triplets.

Le script suivant présente les principaux éléments de cette ontologie :

```
@prefix ont:    <http://example.org/ontology/> .  
@prefix schema: <http://example.org/schema/> .  
@prefix geo:    <http://www.w3.org/2003/01/geo/wgs84_pos#> .  
@prefix rdfs:   <http://www.w3.org/2000/01/rdf-schema#> .  
@prefix rdf:    <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
```

```
#####
```

```
# Classes
```

```
#####
```

```
ont:Location a rdfs:Class ;  
    rdfs:label "Location" .
```

```
ont:City a rdfs:Class ;  
    rdfs:subClassOf ont:Location ;  
    rdfs:label "City" .
```

```
ont:Airport a rdfs:Class ;
```

```

    rdfs:subClassOf ont:Location ;
    rdfs:label "Airport" .

ont:Country a rdfs:Class ;
    rdfs:label "Country" .

#####
# Properties et Contraintes
#####

ont:spatiallyLocatedIn a rdf:Property ;
    rdfs:domain ont:Location ;
    rdfs:range  ont:Location ;
    rdfs:label  "spatially located in (generic)" .

schema:locatedIn a rdf:Property ;
    rdfs:subPropertyOf ont:spatiallyLocatedIn ;
    rdfs:label "located in" .

geo:lat rdfs:domain ont:Location .
geo:long rdfs:domain ont:Location .

```

Cette ontologie permet notamment d'inférer automatiquement qu'une entité de type `City` ou `Airport` est également une instance de `Location`, ainsi que d'inférer le type `Location` à partir de l'utilisation des propriétés géographiques.

6.3 Application du raisonnement RDFS

Le raisonnement est ainsi matérialisé directement dans un nouveau fichier RDF, `data_rdfs.ttl`, qui contient à la fois les triplets d'origine et les triplets inférés. Ce graphe enrichi peut ensuite être interrogé efficacement, sans nécessiter d'activation dynamique du raisonneur lors des requêtes. Le raisonnement RDFS est appliqué à l'aide du raisonneur intégré dans Apache Jena. Le graphe `data_raw.ttl` est combiné avec l'ontologie afin de produire un graphe enrichi contenant les triplets inférés.

La commande utilisée est la suivante :

```

riot --rdfs=rdfs/ontology.ttl \
    data_raw.ttl \
    > data_rdfs.ttl

```

Le fichier `data_rdfs.ttl` constitue le graphe final enrichi. Il contient à la fois les triplets explicitement définis et les triplets déduits automatiquement par raisonnement RDFS. Ce graphe est utilisé comme base pour l'exécution des requêtes SPARQL.

6.4 Optimisation des performances avec Jena TDB2

Le graphe RDF enrichi obtenu après raisonnement (`data_rdfs.ttl`) atteint une taille d'environ 1,8 Go. L'exécution directe de requêtes SPARQL sur un tel fichier, en utilisant l'option `-data` de l'outil `sparql` de Jena, entraîne un temps d'exécution élevé, car le fichier doit être relu et analysé intégralement à chaque requête.

La commande suivante illustre l'exécution directe d'une requête sur le fichier Turtle :

```
sparql --data=data_rdfs.ttl --query=queries/part_4/query_spatiallyLocatedIn.rq
```

Dans ce mode d'exécution, le temps de réponse observé est de l'ordre de plusieurs dizaines de secondes, voire davantage selon les requêtes, en raison du chargement répété du graphe RDF.

Afin d'améliorer significativement les performances, les données ont été chargées dans une base persistante Jena TDB2, qui permet d'indexer le graphe RDF et d'exécuter les requêtes de manière efficace.

Le chargement initial des données dans la base TDB2 est effectué une seule fois à l'aide de la commande suivante :

```
tdb2.tdbloader --loc tdb2-database data_rdfs.ttl
```

Une fois la base créée, les requêtes SPARQL sont exécutées à l'aide de l'outil `tdb2.tdbquery` :

```
tdb2.tdbquery --loc tdb2-database --query queries/part_4/query_spatiallyLocatedIn.rq
```

Dans ce mode, le temps de réponse est réduit à quelques secondes, grâce à l'indexation persistante des données. Cette comparaison met en évidence l'intérêt de l'utilisation d'un moteur de stockage adapté pour l'interrogation de graphes RDF de grande taille.

7 Requêtes SPARQL

Cette section présente un ensemble de requêtes SPARQL permettant d'interroger le graphe RDF construit au cours du projet. Les requêtes exploitent conjointement les différents jeux de données intégrés (villes, aéroports, pays, alignement DBpedia), ainsi que les mécanismes de raisonnement RDFS.

Pour des raisons de lisibilité, les requêtes ne sont pas reproduites intégralement dans le rapport. Chaque requête est référencée par son nom de fichier, accompagnée de son objectif et d'une capture d'écran illustrant les résultats obtenus lors de son exécution.

7.1 Requête fédérée

- `queries/06_federated/query_1_federated.rq` Requête fédérée combinant les données locales avec le point d'accès SPARQL public de DBpedia afin de récupérer les descriptions textuelles (`dbo:abstract`) des pays.

Problème rencontré : Le service est toujours timeout, le script tourne dans le vide.

```
rabah@rabah:~/Projets/rdf-homework$ sparql --data=cities/cities.ttl --data=countries/countries_enriched.ttl --data=alignement/auto_alignement_from_enriched.ttl
--query=queries/part_3/query_1_federated.rq --time
23:14:47 WARN riot :: [line: 32740, col: 6 ] Unicode replacement character U+FFFD in prefixed name
23:14:47 WARN riot :: [line: 32740, col: 7 ] Unicode replacement character U+FFFD in prefixed name
23:14:47 WARN riot :: [line: 33653, col: 11] Unicode replacement character U+FFFD in prefixed name
23:14:47 WARN riot :: [line: 33653, col: 12] Unicode replacement character U+FFFD in prefixed name
23:14:47 WARN riot :: [line: 37690, col: 17] Unicode replacement character U+FFFD in prefixed name
23:14:47 WARN riot :: [line: 37690, col: 18] Unicode replacement character U+FFFD in prefixed name
23:14:47 WARN riot :: [line: 39161, col: 7 ] Unicode replacement character U+FFFD in prefixed name
23:14:47 WARN riot :: [line: 39161, col: 8 ] Unicode replacement character U+FFFD in prefixed name
23:14:47 WARN riot :: [line: 49838, col: 11] Unicode replacement character U+FFFD in prefixed name
23:14:47 WARN riot :: [line: 49838, col: 12] Unicode replacement character U+FFFD in prefixed name
23:14:47 WARN riot :: [line: 51948, col: 10] Unicode replacement character U+FFFD in prefixed name
23:14:47 WARN riot :: [line: 51948, col: 11] Unicode replacement character U+FFFD in prefixed name
23:14:47 WARN riot :: [line: 62325, col: 8 ] Unicode replacement character U+FFFD in prefixed name
23:14:47 WARN riot :: [line: 62325, col: 9 ] Unicode replacement character U+FFFD in prefixed name
23:14:48 WARN riot :: [line: 86690, col: 16] Unicode replacement character U+FFFD in prefixed name
23:14:48 WARN riot :: [line: 86690, col: 17] Unicode replacement character U+FFFD in prefixed name
23:14:49 WARN riot :: [line: 235569, col: 7 ] Unicode replacement character U+FFFD in prefixed name
23:14:49 WARN riot :: [line: 235569, col: 8 ] Unicode replacement character U+FFFD in prefixed name
23:14:49 WARN riot :: [line: 271386, col: 7 ] Unicode replacement character U+FFFD in prefixed name
23:14:49 WARN riot :: [line: 271386, col: 8 ] Unicode replacement character U+FFFD in prefixed name
23:14:49 WARN riot :: [line: 281759, col: 8 ] Unicode replacement character U+FFFD in prefixed name
23:14:49 WARN riot :: [line: 281759, col: 9 ] Unicode replacement character U+FFFD in prefixed name
23:14:50 WARN riot :: [line: 329992, col: 7 ] Unicode replacement character U+FFFD in prefixed name
23:14:50 WARN riot :: [line: 329992, col: 8 ] Unicode replacement character U+FFFD in prefixed name
23:14:50 WARN riot :: [line: 354316, col: 12] Unicode replacement character U+FFFD in prefixed name
23:14:50 WARN riot :: [line: 354316, col: 13] Unicode replacement character U+FFFD in prefixed name
[]
```

FIGURE 4 – Résultats de la requête fédérée interrogeant DBpedia via la clause `SERVICE`

7.2 Requêtes avec expressions de chemin

- queries/03_paths/query_5_path.rq Utilisation d'une expression de chemin SPARQL pour identifier des aéroports situés dans une même ville à partir de relations de localisation.

airport1	airport2	city
<http://example.org/resource/airport_MI37>	<http://example.org/resource/airport_K76G>	<http://example.org/resource/city_marine_city>
<http://example.org/resource/airport_MI37>	<http://example.org/resource/airport_59MI>	<http://example.org/resource/city_marine_city>
<http://example.org/resource/airport_MI37>	<http://example.org/resource/airport_20MI>	<http://example.org/resource/city_marine_city>
<http://example.org/resource/airport_US-12351>	<http://example.org/resource/airport_KHEF>	<http://example.org/resource/city_manassas>
<http://example.org/resource/airport_US-12351>	<http://example.org/resource/airport_52VA>	<http://example.org/resource/city_manassas>
<http://example.org/resource/airport_US-12351>	<http://example.org/resource/airport_2VG6>	<http://example.org/resource/city_manassas>
<http://example.org/resource/airport_US-12351>	<http://example.org/resource/airport_66VA>	<http://example.org/resource/city_manassas>
<http://example.org/resource/airport_US-12351>	<http://example.org/resource/airport_51VA>	<http://example.org/resource/city_manassas>
<http://example.org/resource/airport_US-12351>	<http://example.org/resource/airport_27VA>	<http://example.org/resource/city_manassas>
<http://example.org/resource/airport_US-9433>	<http://example.org/resource/airport_US-11325>	<http://example.org/resource/city_joliet>
<http://example.org/resource/airport_US-9433>	<http://example.org/resource/airport_KJOT>	<http://example.org/resource/city_joliet>
<http://example.org/resource/airport_US-9433>	<http://example.org/resource/airport_IL98>	<http://example.org/resource/city_joliet>
<http://example.org/resource/airport_US-9433>	<http://example.org/resource/airport_61S9>	<http://example.org/resource/city_joliet>
<http://example.org/resource/airport_ENWR>	<http://example.org/resource/airport_ENWA>	<http://example.org/resource/city_north_sea>
<http://example.org/resource/airport_ENWR>	<http://example.org/resource/airport_ENQM>	<http://example.org/resource/city_north_sea>
<http://example.org/resource/airport_ENWR>	<http://example.org/resource/airport_ENMK>	<http://example.org/resource/city_north_sea>
<http://example.org/resource/airport_ENWR>	<http://example.org/resource/airport_DE-0915>	<http://example.org/resource/city_north_sea>
<http://example.org/resource/airport_ENWR>	<http://example.org/resource/airport_ENWS>	<http://example.org/resource/city_north_sea>
<http://example.org/resource/airport_ENWR>	<http://example.org/resource/airport_ENJS>	<http://example.org/resource/city_north_sea>
<http://example.org/resource/airport_ENWR>	<http://example.org/resource/airport_ENWV>	<http://example.org/resource/city_north_sea>

FIGURE 5 – Résultats de la requête utilisant une expression de chemin SPARQL

7.3 Requêtes utilisant OPTIONAL

- queries/04_optional/query_2_optional.rq Illustration de l'utilisation de la clause OPTIONAL pour récupérer les coordonnées géographiques des aéroports lorsque certaines informations sont absentes.

airport	name	type	lat	long	city
<http://example.org/resource/airport_00A>	"Total RF Heliport"	"heliport"	40.070985		<http://example.org/resource/ci
<http://example.org/resource/airport_00AA>	"Aero B Ranch Airport"	"small airport"			<http://example.org/resource/ci
<http://example.org/resource/airport_00AK>	"Lowell Field"	"small airport"	59.947733	-151.692524	<http://example.org/resource/ci
<http://example.org/resource/airport_00AL>	"Epps Airpark"	"small airport"	34.86479949951172	-86.77030181884766	<http://example.org/resource/ci
<http://example.org/resource/airport_00AN>	"Katmai Lodge Airport"	"small airport"	59.093287	-156.456699	<http://example.org/resource/ci
<http://example.org/resource/airport_00AS>	"Fulton Airport"	"small airport"	34.9428028	-97.8180194	<http://example.org/resource/ci
<http://example.org/resource/airport_00AZ>	"Cordes Airport"	"small airport"	34.305599212646484	-112.16500091552734	<http://example.org/resource/ci
<http://example.org/resource/airport_00CA>	"Goldstone (GTS) Airport"	"small airport"	35.35474	-116.885329	<http://example.org/resource/ci
<http://example.org/resource/airport_00CL>	"Williams Ag Airport"	"small airport"	39.427188	-121.763427	<http://example.org/resource/ci
<http://example.org/resource/airport_00CN>	"Kitchen Creek Helibase Heliport"	"heliport"	32.7273736	-116.4597417	<http://example.org/resource/ci
<http://example.org/resource/airport_00FA>	"Grass Patch Airport"	"small airport"	28.64550018310547	-82.21900177001953	<http://example.org/resource/ci
<http://example.org/resource/airport_00FL>	"River Oak Airport"	"small airport"	27.230899810791016	-80.96920013427734	<http://example.org/resource/ci
<http://example.org/resource/airport_00GA>	"Lt World Airport"	"small airport"	33.76750183105469	-84.06829833984375	<http://example.org/resource/ci
<http://example.org/resource/airport_00GE>	"Caffrey Heliport"	"heliport"	33.887982	-84.736983	<http://example.org/resource/ci
<http://example.org/resource/airport_00HI>	"Kaupulehu Heliport"	"heliport"	19.832881	-155.978347	<http://example.org/resource/ci
<http://example.org/resource/airport_00ID>	"Delta Shores Airport"	"small airport"	48.145301818847656	-116.21399688720703	<http://example.org/resource/ci
<http://example.org/resource/airport_00IG>	"Goltl Airport"	"small airport"	39.724028	-101.395994	<http://example.org/resource/ci
<http://example.org/resource/airport_00IL>	"Hammer Airport"	"small airport"	41.978401	-89.560402	<http://example.org/resource/ci
<http://example.org/resource/airport_00IN>	"St Mary Medical Center Heliport"	"heliport"	41.51139831542969	-87.2605972290039	<http://example.org/resource/ci
<http://example.org/resource/airport_00IS>	"Hayenga's Cant Find Farms Airport"	"small airport"	40.02560043334961	-89.1229019165039	<http://example.org/resource/ci

FIGURE 6 – Résultats de la requête utilisant la clause OPTIONAL

7.4 Requêtes sur graphes nommés

- `queries/05_named_graphs/query_3_named_graphs.rq` Interrogation explicite d'un graphe nommé correspondant à un fichier RDF spécifique (`cities.ttl`).

city	lat	long
<http://example.org/resource/Charlotte_Amalie>	18.3420	-64.9331
<http://example.org/resource/Lupane>	-18.9315	27.8070
<http://example.org/resource/Nelspruit>	-25.4745	30.9703
<http://example.org/resource/Al_Jabīn>	14.7040	43.5990
<http://example.org/resource/%E2%80%98Amr%C4%81n>	15.6594	43.9439
<http://example.org/resource/Satupa%E2%80%98itea>	-13.7659	-172.3269
<http://example.org/resource/Mulifanua>	-13.8333	-171.9833
<http://example.org/resource/Safotulafai>	-13.6817	-172.1850
<http://example.org/resource/Saratamata>	-15.2875	167.9903
<http://example.org/resource/Đà_Ni ng>	16.0748	108.2240
<http://example.org/resource/Kon_Tum>	14.3545	108.0076
<http://example.org/resource/Thành_Phố_Bà_Ri a>	10.4963	107.1685
<http://example.org/resource/Tüytepa>	41.0428	69.3572
<http://example.org/resource/Kalungu>	-0.1681	31.7600
<http://example.org/resource/Rubanda>	-1.1864	29.8433
<http://example.org/resource/Nabilatuk>	2.0525	34.5734
<http://example.org/resource/Kole>	2.4286	32.8011
<http://example.org/resource/Leju>	3.1859	31.0610
<http://example.org/resource/Kibingo>	-0.5700	30.4150
<http://example.org/resource/Palenga>	2.5764	32.3544

FIGURE 7 – Résultats de la requête SPARQL sur graphe nommé

7.5 Requêtes avec filtrage avancé

- `queries/06_filter/query_6_filter_not_exists.rq` Utilisation de la clause `FILTER NOT EXISTS` pour identifier des entités ne satisfaisant pas une condition donnée (par exemple, aéroports sans coordonnées géographiques).

```
-----  
| airport |  
=====
```

<http://example.org/resource/airport_00A>	
<http://example.org/resource/airport_00AA>	

```
-----
```

FIGURE 8 – Résultats de la requête utilisant `FILTER NOT EXISTS`

7.6 Requêtes avec raisonnement RDFS

Avant raisonnement RDFS

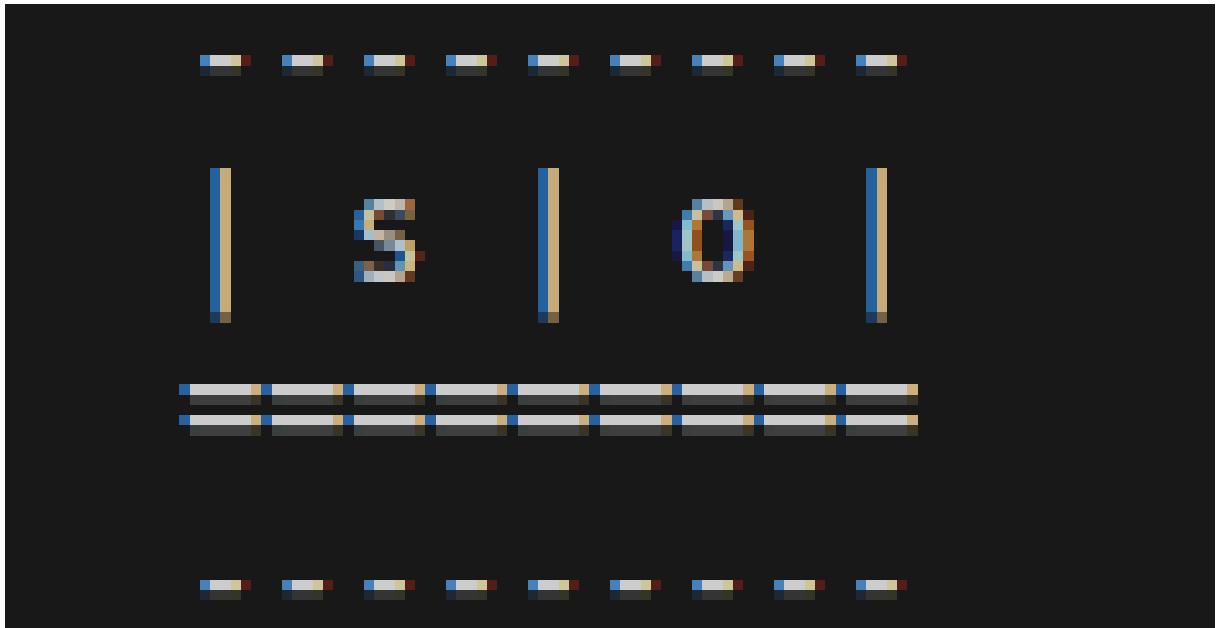


FIGURE 9 – Résultat de la requête `query_spatiallyLocatedIn.rq` avant application du raisonnement RDFS

Après raisonnement RDFS

s	o
<http://example.org/resource/airport_JP-1655>	<http://example.org/resource/city_shiranuka>
<http://example.org/resource/airport_JP-1657>	<http://example.org/resource/city_toyohashi>
<http://example.org/resource/airport_JP-1676>	<http://example.org/resource/city_toyohashi>
<http://example.org/resource/airport_JP-1677>	<http://example.org/resource/city_toyohashi>
<http://example.org/resource/airport_JP-2171>	<http://example.org/resource/city_toyohashi>

s	o
<http://example.org/resource/Tokyo>	<http://example.org/resource/Japan>
<http://example.org/resource/Ôsaka>	<http://example.org/resource/Japan>
<http://example.org/resource/Nagoya>	<http://example.org/resource/Japan>
<http://example.org/resource/Yokohama>	<http://example.org/resource/Japan>
<http://example.org/resource/Fukuoka>	<http://example.org/resource/Japan>

FIGURE 10 – Résultat de la requête `query_spatiallyLocatedIn.rq` après application du raisonnement RDFS

8 Conclusion

Ce projet a permis de mettre en œuvre les principaux concepts du Web sémantique à travers la construction d'un graphe RDF intégré à partir de jeux de données hétérogènes décrivant des villes, des pays et des aéroports. Les données, initialement disponibles sous des formats variés (CSV et RDF), ont été transformées, enrichies, alignées et intégrées au sein d'un graphe unique interrogeable via SPARQL.

Les requêtes SPARQL développées illustrent différentes capacités du langage, notamment l'agrégation, l'utilisation de `OPTIONAL`, les expressions de chemin, l'interrogation de graphes nommés et les requêtes fédérées via le mécanisme `SERVICE`.

Le raisonnement RDFS, appliqué en amont et matérialisé dans le graphe enrichi, a montré son intérêt pour inférer de nouveaux triplets sans modifier les données sources. Il permet notamment de généraliser les requêtes et de raisonner sur des concepts abstraits.

Toutefois, le raisonnement RDFS présente certaines limites : il reste volontairement simple, ne permet pas d'exprimer des contraintes complexes, et ne prend pas en charge des règles logiques avancées comme celles offertes par **OWL** ou les moteurs de règles dédiés.

Plusieurs améliorations sont envisageables pour faire évoluer ce projet. Une première piste consisterait à enrichir l'ontologie à l'aide de schémas plus expressifs, en introduisant par exemple des propriétés inverses ou des restrictions de cardinalité. L'intégration de nouveaux jeux de données, tels que des données démographiques détaillées, des réseaux de transport ou des indicateurs socio-économiques, permettrait également d'augmenter la richesse du graphe et la pertinence des requêtes.

Enfin, une évolution prometteuse du projet consisterait à coupler le graphe RDF avec des techniques **d'apprentissage automatique**. Par exemple, les données intégrées pourraient être utilisées pour entraîner des modèles de prédiction ou de classification, tels que l'estimation de **l'attractivité d'une ville** ou **la détection d'anomalies dans les infrastructures aéroportuaires**. Dans ce contexte, le graphe RDF servirait de base de connaissances structurée, garantissant la cohérence sémantique des données d'entrée et facilitant l'interprétabilité des modèles d'apprentissage.

Ce projet illustre ainsi la complémentarité entre le Web sémantique et les approches modernes de traitement des données, en montrant comment des graphes RDF enrichis peuvent constituer un socle solide pour des applications analytiques et décisionnelles avancées.

Bibliographie

- Apache Software Foundation. *Apache Jena – A Java framework for building Semantic Web and Linked Data applications*. Documentation officielle. <https://jena.apache.org/>
- Apache Software Foundation. *Tarql – SPARQL for Tables*. Outil de transformation de données tabulaires vers RDF. <https://tarql.github.io/>
- Apache Software Foundation. *Apache Jena TDB2 – High performance RDF database*. Documentation officielle. <https://jena.apache.org/documentation/tdb2/>
- Kaggle. *Kaggle Datasets – Open datasets platform*. Plateforme de diffusion de jeux de données ouverts. <https://www.kaggle.com/datasets>
- DBpedia Association. *DBpedia – A large-scale, multilingual knowledge base extracted from Wikipedia*. Documentation officielle. <https://www.dbpedia.org/>
- Wikidata Contributors. *Wikidata – A free and open knowledge base*. Fondation Wikimedia. <https://www.wikidata.org/>
- World Wide Web Consortium (W3C). *RDF 1.1 Concepts and Abstract Syntax*. Recommandation W3C. <https://www.w3.org/TR/rdf11-concepts/>
- World Wide Web Consortium (W3C). *RDF Schema 1.1*. Recommandation W3C. <https://www.w3.org/TR/rdf-schema/>
- World Wide Web Consortium (W3C). *SPARQL 1.1 Query Language*. Recommandation W3C. <https://www.w3.org/TR/sparql11-query/>